

Feature Separation Analysis

```
feature_separation_current <- sdf_sql(
  sc,
  "
SELECT
  label_3m,
  COUNT(*) AS parties,

  /* -----
  Deterioration
  ----- */
  AVG(deterioration_acceleration)
  AS deterioration_mean,

  percentile_approx(
    deterioration_acceleration, 0.25
  ) AS deterioration_p25,

  percentile_approx(
    deterioration_acceleration, 0.50
  ) AS deterioration_median,

  percentile_approx(
    deterioration_acceleration, 0.75
  ) AS deterioration_p75,

  /* -----
  Adverse behavior
  ----- */
  AVG(adverse_acceleration)
  AS adverse_mean,

  percentile_approx(
    adverse_acceleration, 0.50
  ) AS adverse_median,

  /* -----
  Max DPD jump
  ----- */
  AVG(max_dpd_jump_recent_3m)
  AS max_dpd_mean,

  percentile_approx(
    max_dpd_jump_recent_3m, 0.50
  ) AS max_dpd_median,
```

```

percentile_approx(
    max_dpd_jump_recent_3m, 0.95
) AS max_dpd_p95,

/* -----
   Active accounts
   ----- */
AVG(active_accounts_change_vs_baseline)
    AS active_change_mean,

percentile_approx(
    active_accounts_change_vs_baseline, 0.25
) AS active_change_p25,

percentile_approx(
    active_accounts_change_vs_baseline, 0.50
) AS active_change_median,

percentile_approx(
    active_accounts_change_vs_baseline, 0.75
) AS active_change_p75,

AVG(active_accounts_changed_flag * 1.0)
    AS active_changed_pct,

/* -----
   Balance
   ----- */
AVG(balance_change_model)
    AS balance_change_mean,

percentile_approx(
    balance_change_model, 0.50
) AS balance_change_median,

AVG(log1p_abs_balance_change_model)
    AS logabs_balance_mean,

percentile_approx(
    log1p_abs_balance_change_model, 0.50
) AS logabs_balance_median,

AVG(balance_change_available_flag * 1.0)
    AS balance_available_pct,

```

```

/* -----
Past due
----- */
AVG(past_due_change_model)
  AS pastdue_change_mean,

percentile_approx(
  past_due_change_model, 0.50
) AS pastdue_change_median,

AVG(past_due_change_available_flag * 1.0)
  AS pastdue_available_pct,


/* -----
Delinquency
----- */
AVG(delinquency_share_change_model)
  AS delinquency_change_mean,

percentile_approx(
  delinquency_share_change_model, 0.50
) AS delinquency_change_median,

AVG(delinquency_change_available_flag * 1.0)
  AS delinquency_available_pct,


/* -----
Utilization
----- */
AVG(avg_utilization_change_model)
  AS avg_util_change_mean,

percentile_approx(
  avg_utilization_change_model, 0.50
) AS avg_util_change_median,

AVG(weighted_utilization_change_model)
  AS weighted_util_change_mean,

percentile_approx(
  weighted_utilization_change_model, 0.50
) AS weighted_util_change_median,

AVG(utilization_change_available_flag * 1.0)
  AS utilization_available_pct

```

```
FROM bu_frd.model_features_ab_final_2022_2025
```

```
GROUP BY label_3m
```

```
ORDER BY label_3m
```

```
"
```

```
) %>%
```

```
collect()
```

```
feature_separation_current
```

2- Separación de las features Z

```
feature_separation_z <- sdf_sql(
```

```
sc,
```

```
"
```

```
SELECT
```

```
label_3m,
```

```
COUNT(*) AS parties,
```

```
/* Active account Z */
```

```
AVG(active_accounts_z_model)
```

```
AS active_z_mean,
```

```
percentile_approx(
```

```
active_accounts_z_model, 0.25
```

```
) AS active_z_p25,
```

```
percentile_approx(
```

```
active_accounts_z_model, 0.50
```

```
) AS active_z_median,
```

```
percentile_approx(
```

```
active_accounts_z_model, 0.75
```

```
) AS active_z_p75,
```

```
AVG(active_accounts_z_available_flag * 1.0)
```

```
AS active_z_available_pct,
```

```
/* Average utilization Z */
```

```
AVG(avg_utilization_z_model)
```

```
AS avg_util_z_mean,
```

```
percentile_approx(
```

```
avg_utilization_z_model, 0.25
```

```

) AS avg_util_z_p25,

percentile_approx(
    avg_utilization_z_model, 0.50
) AS avg_util_z_median,

percentile_approx(
    avg_utilization_z_model, 0.75
) AS avg_util_z_p75,

AVG(avg_utilization_z_available_flag * 1.0)
    AS avg_util_z_available_pct,

/* Weighted utilization Z */
AVG(weighted_utilization_z_model)
    AS weighted_util_z_mean,

percentile_approx(
    weighted_utilization_z_model, 0.25
) AS weighted_util_z_p25,

percentile_approx(
    weighted_utilization_z_model, 0.50
) AS weighted_util_z_median,

percentile_approx(
    weighted_utilization_z_model, 0.75
) AS weighted_util_z_p75,

AVG(weighted_utilization_z_available_flag * 1.0)
    AS weighted_util_z_available_pct

FROM bu_frd.model_features_ab_final_2022_2025

GROUP BY label_3m

ORDER BY label_3m
"
) %>%
collect()

feature_separation_z

```

4. Availability vs Value Signal

```
feature_availability_signal <- sdf_sql(  
  sc,  
  "  
  SELECT  
    feature,  
    availability_status,  
  
    COUNT(*) AS parties,  
  
    SUM(label_3m) AS positive_cases,  
  
    AVG(label_3m * 1.0)  
    AS positive_rate  
  
  FROM (  
  
    /* Balance */  
    SELECT  
      label_3m,  
      'balance_change' AS feature,  
      CASE  
        WHEN balance_change_available_flag = 1  
        THEN 'AVAILABLE'  
        ELSE 'NOT_AVAILABLE'  
      END AS availability_status  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
    UNION ALL  
  
    /* Past due */  
    SELECT  
      label_3m,  
      'past_due_change',  
      CASE  
        WHEN past_due_change_available_flag = 1  
        THEN 'AVAILABLE'  
        ELSE 'NOT_AVAILABLE'  
      END  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
    UNION ALL  
  
    /* Delinquency */  
    SELECT
```

```

label_3m,
'delinquency_change',
CASE
    WHEN delinquency_change_available_flag = 1
    THEN 'AVAILABLE'
    ELSE 'NOT_AVAILABLE'
END

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

```

```

/* Utilization */
SELECT
    label_3m,
    'utilization_change',
CASE
    WHEN utilization_change_available_flag = 1
    THEN 'AVAILABLE'
    ELSE 'NOT_AVAILABLE'
END

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

```

```

/* Active accounts Z */
SELECT
    label_3m,
    'active_accounts_z',
CASE
    WHEN active_accounts_z_available_flag = 1
    THEN 'AVAILABLE'
    ELSE 'NOT_AVAILABLE'
END

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

```

```

/* Average utilization Z */
SELECT
    label_3m,
    'avg_utilization_z',
CASE
    WHEN avg_utilization_z_available_flag = 1
    THEN 'AVAILABLE'
    ELSE 'NOT_AVAILABLE'

```

```

        END

        FROM bu_frd.model_features_ab_final_2022_2025

        UNION ALL

        /* Weighted utilization Z */
        SELECT
            label_3m,
            'weighted_utilization_z',
            CASE
                WHEN weighted_utilization_z_available_flag = 1
                THEN 'AVAILABLE'
                ELSE 'NOT_AVAILABLE'
            END

        FROM bu_frd.model_features_ab_final_2022_2025

    ) x

    GROUP BY
        feature,
        availability_status

    ORDER BY
        feature,
        availability_status
    "
) %>%
collect()

feature_availability_signal

```

Feature Shape / Non-Linearity

```

feature_shape_categorical <- sdf_sql(
  sc,
  "
  WITH long_features AS (

    /* Active account change */
    SELECT
      label_3m,
      'active_accounts_change' AS feature,

```

```

CASE
  WHEN active_accounts_change_vs_baseline < 0
    THEN 'NEGATIVE'
  WHEN active_accounts_change_vs_baseline > 0
    THEN 'POSITIVE'
  ELSE 'ZERO'
END AS feature_group

```

```

FROM bu_frd.model_features_ab_final_2022_2025

```

```

UNION ALL

```

```

/* Active accounts changed flag */

```

```

SELECT
  label_3m,
  'active_accounts_changed_flag',

```

```

CASE
  WHEN active_accounts_changed_flag = 1
    THEN 'CHANGED'
  ELSE 'NO_CHANGE'
END

```

```

FROM bu_frd.model_features_ab_final_2022_2025

```

```

UNION ALL

```

```

/* Deterioration */

```

```

SELECT
  label_3m,
  'deterioration_acceleration',

```

```

CASE
  WHEN deterioration_acceleration < 0
    THEN 'NEGATIVE'
  WHEN deterioration_acceleration > 0
    THEN 'POSITIVE'
  ELSE 'ZERO'
END

```

```

FROM bu_frd.model_features_ab_final_2022_2025

```

```

UNION ALL

```

```

/* Adverse */

```

```

SELECT
    label_3m,
    'adverse_acceleration',

    CASE
        WHEN adverse_acceleration < 0
            THEN 'NEGATIVE'
        WHEN adverse_acceleration > 0
            THEN 'POSITIVE'
        ELSE 'ZERO'
    END

FROM bu_frd.model_features_ab_final_2022_2025

```

UNION ALL

```

/* Max DPD */
SELECT
    label_3m,
    'max_dpd_jump_recent_3m',

    CASE
        WHEN max_dpd_jump_recent_3m = 0
            THEN '0'
        WHEN max_dpd_jump_recent_3m = 1
            THEN '1'
        ELSE '2+'
    END

FROM bu_frd.model_features_ab_final_2022_2025

```

UNION ALL

```

/* Past due */
SELECT
    label_3m,
    'past_due_change',

    CASE
        WHEN past_due_change_available_flag = 0
            THEN 'MISSING'
        WHEN past_due_change_model < 0
            THEN 'NEGATIVE'
        WHEN past_due_change_model > 0
            THEN 'POSITIVE'
        ELSE 'ZERO'
    END

```

END

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

/* Delinquency */

SELECT

label_3m,
'delinquency_change',

CASE

WHEN delinquency_change_available_flag = 0
THEN 'MISSING'

WHEN delinquency_share_change_model < 0
THEN 'NEGATIVE'

WHEN delinquency_share_change_model > 0
THEN 'POSITIVE'

ELSE 'ZERO'

END

FROM bu_frd.model_features_ab_final_2022_2025

)

SELECT

feature,
feature_group,

COUNT(*) AS parties,

SUM(label_3m) AS positive_cases,

AVG(label_3m * 1.0) AS positive_rate,

AVG(label_3m * 1.0) / (62599.0 / 688589.0)
AS lift

FROM long_features

GROUP BY

feature,
feature_group

ORDER BY

feature,
feature_group

"

```
) %>%  
  collect()
```

feature_shape_categorical

Continuous features por deciles

```
feature_shape_continuous <- sdf_sql(  
  sc,  
  "  
  WITH long_features AS (  
  
    /* Balance signed */  
    SELECT  
      label_3m,  
      'balance_change' AS feature,  
      balance_change_model AS feature_value  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
    WHERE balance_change_available_flag = 1  
  
    UNION ALL  
  
    /* Balance magnitude */  
    SELECT  
      label_3m,  
      'log1p_abs_balance_change',  
      log1p_abs_balance_change_model  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
    WHERE balance_change_available_flag = 1  
  
    UNION ALL  
  
    /* Average utilization change */  
    SELECT  
      label_3m,  
      'avg_utilization_change',  
      avg_utilization_change_model  
  
    FROM bu_frd.model_features_ab_final_2022_2025
```

WHERE utilization_change_available_flag = 1

UNION ALL

/* Weighted utilization change */

SELECT

label_3m,
'weighted_utilization_change',
weighted_utilization_change_model

FROM bu_frd.model_features_ab_final_2022_2025

WHERE utilization_change_available_flag = 1

UNION ALL

/* Active account Z */

SELECT

label_3m,
'active_accounts_z',
active_accounts_z_model

FROM bu_frd.model_features_ab_final_2022_2025

WHERE active_accounts_z_available_flag = 1

UNION ALL

/* Average utilization Z */

SELECT

label_3m,
'avg_utilization_z',
avg_utilization_z_model

FROM bu_frd.model_features_ab_final_2022_2025

WHERE avg_utilization_z_available_flag = 1

UNION ALL

/* Weighted utilization Z */

SELECT

label_3m,

```

        'weighted_utilization_z',
        weighted_utilization_z_model

FROM bu_frd.model_features_ab_final_2022_2025

WHERE weighted_utilization_z_available_flag = 1
),

deciles AS (

SELECT
    *,
    NTILE(10) OVER (
        PARTITION BY feature
        ORDER BY feature_value
    ) AS feature_decile

FROM long_features
)

SELECT
    feature,
    feature_decile,

    COUNT(*) AS parties,

    SUM(label_3m) AS positive_cases,

    AVG(label_3m * 1.0) AS positive_rate,

    AVG(label_3m * 1.0) / (62599.0 / 688589.0)
    AS lift,

    MIN(feature_value) AS min_value,
    percentile_approx(feature_value, 0.50)
    AS median_value,
    MAX(feature_value) AS max_value

FROM deciles

GROUP BY
    feature,
    feature_decile

ORDER BY
    feature,
    feature_decile
"

```

```
) %>%  
  collect()
```

feature_shape_continuous

3- Multi-Signal Convergence

```
feature_signal_convergence <- sdf_sql(  
  sc,  
  "  
  WITH signals AS (  
  
    SELECT  
      party_id,  
      label_3m,  
  
      /* 1. Active account structure changed */  
      CASE  
        WHEN active_accounts_changed_flag = 1  
        THEN 1 ELSE 0  
      END AS signal_active_change,  
  
      /* 2. Material balance movement */  
      CASE  
        WHEN balance_change_available_flag = 1  
        AND log1p_abs_balance_change_model >= 7.72  
        THEN 1 ELSE 0  
      END AS signal_balance_magnitude,  
  
      /* 3. Balance information available */  
      CASE  
        WHEN balance_change_available_flag = 1  
        THEN 1 ELSE 0  
      END AS signal_balance_available,  
  
      /* 4. Utilization information available */  
      CASE  
        WHEN utilization_change_available_flag = 1  
        THEN 1 ELSE 0  
      END AS signal_util_available,  
  
      /* 5. Active Z history available */  
      CASE  
        WHEN active_accounts_z_available_flag = 1  
        THEN 1 ELSE 0  
      END AS signal_active_z_available
```

```
FROM bu_frd.model_features_ab_final_2022_2025
),
```

```
scored AS (
```

```
SELECT
```

```
*,
```

```
    signal_active_change
  + signal_balance_magnitude
  + signal_balance_available
  + signal_util_available
  + signal_active_z_available
  AS signal_count
```

```
FROM signals
```

```
)
```

```
SELECT
```

```
    signal_count,
```

```
    COUNT(*) AS parties,
```

```
    SUM(label_3m) AS positive_cases,
```

```
    AVG(label_3m * 1.0) AS positive_rate,
```

```
    AVG(label_3m * 1.0) /
    (62599.0 / 688589.0) AS lift,
```

```
    SUM(
        CASE WHEN label_3m = 0 THEN 1 ELSE 0 END
    ) AS negative_parties,
```

```
    SUM(
        CASE WHEN label_3m = 1 THEN 1 ELSE 0 END
    ) AS positive_parties
```

```
FROM scored
```

```
GROUP BY signal_count
```

```
ORDER BY signal_count
```

```
"
```

```
) %>%
```

```
collect()
```

```
feature_signal_convergence
```

descriptivo del convergence analysis

```
feature_signal_convergence <- feature_signal_convergence %>%  
  mutate(  
    pct_of_all =  
      parties / sum(parties),  
  
    pct_of_negatives =  
      negative_parties / sum(negative_parties),  
  
    pct_of_positives =  
      positive_parties / sum(positive_parties)  
  )  
  
feature_signal_convergence
```